Proyecto de Bases de Datos

Reviews de Amazon – MySQL, MongoDB y Neo4J

Yago Vallarino Fernández-Cavada
Ignacio Mascarell Agúndez

Curso 2025–2026

Contents

1 Parte 1: Diseño y carga de datos **2**
1.1 Esquema relacional de MySQL . . . 2
1.2 Justificación del reparto MySQL / MongoDB . . . 2
1.3 Ficheros de configuración y carga . . . 3
1.4 Garantías de consistencia . . . 3

2 Parte 2: Visualización de datos **3**
2.1 Gráfica 1: Reviews por año . . . 3
2.2 Gráfica 2: Popularidad de artículos . . . 4
2.3 Gráfica 3: Histograma por nota . . . 4
2.4 Gráfica 4: Evolución temporal por categorías . . . 4
2.5 Gráfica 5: Reviews por usuario . . . 4
2.6 Gráfica 6: Nube de palabras . . . 4
2.7 Gráfica 7: Rating medio por categoría (libre) . . . 4

3 Parte 3: Neo4J **4**
3.1 Apartado 4.1: Similitudes Pearson . . . 4
3.2 Apartado 4.2: Artículos aleatorios . . . 5
3.3 Apartado 4.3: Usuarios con múltiples categorías . . . 5
3.4 Apartado 4.4: Artículos populares . . . 6

4 Parte 4: Nuevo dataset **6**

5 Parte 5: Visualización adicional y Machine Learning **7**
5.1 Gráficas en Tableau . . . 7
5.2 Modelo de Machine Learning para recomendación . . . 8
5.3 Mecanismo de recomendación implementado . . . 9

6 División del trabajo **10**

7 Uso de la Inteligencia Artificial **10**

1 Parte 1: Diseño y carga de datos

1.1 Esquema relacional de MySQL

La base de datos MySQL se compone de tres tablas con las siguientes relaciones:

- **usuarios** (`id_usuario VARCHAR(50)`, `nombre VARCHAR(255)`). Almacena el reviewerID y reviewerName de cada autor de reviews.
- **productos** (`asin VARCHAR(50)`, `categoria VARCHAR(50)`). Almacena el identificador del producto y la categoría a la que pertenece (derivada del fichero de origen).
- **reviews_meta** (`id_review INT AUTO_INCREMENT`, `id_usuario` FK → `usuarios`, `asin` FK → `productos`, `rating FLOAT`, `fecha DATE`, `unix_time BIGINT`). Almacena los metadatos estructurados de cada review.

Las claves foráneas garantizan la integridad referencial: cada review debe pertenecer a un usuario y a un producto existentes. Para evitar duplicados en usuarios y productos, se comprueba la existencia del registro antes de insertarlo.

A continuación se muestra el esquema relacional de la base de datos MySQL:

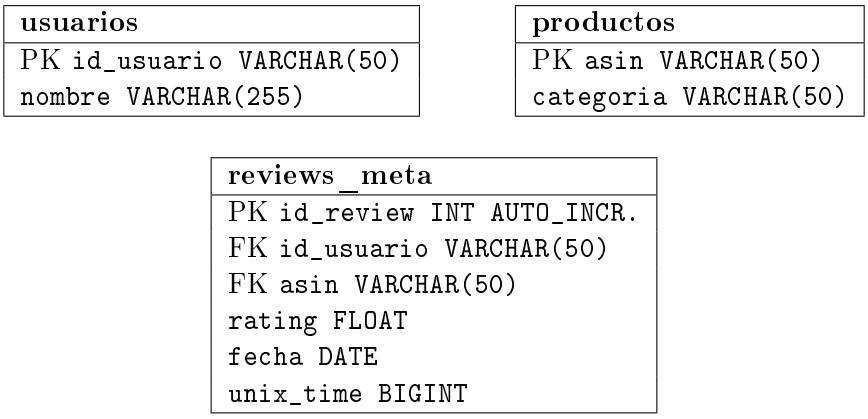


Figure 1: Esquema relacional de la base de datos MySQL.

El diagrama muestra las tres tablas y sus relaciones. La tabla `reviews_meta` referencia a `usuarios` mediante la clave foránea `id_usuario` y a `productos` mediante la clave foránea `asin`. Esta separación permite consultar metadatos estructurados con JOINs eficientes en MySQL, mientras que los campos textuales y semiestructurados se almacenan en MongoDB.

1.2 Justificación del reparto MySQL / MongoDB

Se ha decidido el siguiente reparto de los 9 campos del JSON original:

Campo JSON	Base de datos	Campo almacenado
reviewerID	MySQL	usuarios.id_usuario
reviewerName	MySQL	usuarios.nombre
asin	MySQL	productos.asin + reviews_meta.asin
overall	MySQL	reviews_meta.rating
reviewTime	MongoDB	reviewTime (fecha parseada, tipo Date/BSON Date)
unixReviewTime	MySQL	reviews_meta.unix_time + fecha (DATE generada en la carga)
reviewText	MongoDB	reviewText
summary	MongoDB	summary
helpful	MongoDB	helpful (array)

Razonamiento:

- MySQL almacena los datos estructurados y numéricos que se consultan frecuentemente con JOINs, agregaciones (GROUP BY, COUNT, AVG) y filtros por fecha/categoría. Estos datos se usan en las gráficas de visualización y en las consultas de Neo4J.
- MongoDB almacena los datos textuales y semiestructurados (`reviewText`, `summary`, `helpful`). Estos campos son de longitud variable, no participan en JOINs relacionales y se consultan principalmente para la nube de palabras. MongoDB es ideal para documentos con esquema flexible.
- La fecha utilizada en MySQL se almacena como DATE para permitir consultas temporales, mientras que en MongoDB el campo `reviewTime` se almacena como fecha parseada (tipo Date/BSON Date), cumpliendo el requisito del enunciado de no conservarlo como string. En la implementación actual, la fecha DATE de MySQL se obtiene durante la carga a partir de `unixReviewTime`, y `reviewTime` se parsea en Python antes de insertarse en MongoDB.
- Se añade un campo `id_sql` en el documento MongoDB que referencia al `id_review` de MySQL, permitiendo vincular ambas bases de datos.
- Se añade un campo `categoria` en MongoDB para facilitar el filtrado por categoría en la nube de palabras.

1.3 Ficheros de configuración y carga

- `configuracion.py`: Centraliza todas las rutas de los ficheros, los nombres de las bases de datos y colecciones, los parámetros ajustables (`BATCH_SIZE`, `TOP_N_USUARIOS`) y las variables de conexión de MySQL, MongoDB y Neo4J. El profesor solo necesita modificar este fichero para adaptar rutas y completar, en su caso, las credenciales de acceso.
- `load_data.py`: Ejecutando este fichero se crea la infraestructura completa (base de datos, tablas) y se cargan los 4 datasets. La lectura se realiza línea a línea, sin cargar el fichero completo en memoria. Durante la carga, `unixReviewTime` se transforma en una fecha DATE para MySQL y `reviewTime` se parsea a fecha antes de insertarse en MongoDB.

1.4 Garantías de consistencia

Si la inserción en MongoDB falla tras insertar la review en MySQL, se ejecuta un DELETE compensatorio sobre `reviews_meta` para evitar que quede una review huérfana en la tabla principal. Los commits de MySQL se realizan por lotes y al final de cada categoría. Las conexiones se cierran siempre en bloques `finally`. Este mecanismo garantiza la coherencia de las reviews insertadas, aunque un usuario o producto creados previamente durante ese proceso pueden permanecer almacenados.

2 Parte 2: Visualización de datos

El fichero `menu_visualizacion.py` implementa un menú por terminal con 8 opciones (7 gráficas + salir) que se repite en bucle hasta que el usuario selecciona la opción de salir.

2.1 Gráfica 1: Reviews por año

Histograma de barras mostrando el número de reviews por año. Permite filtrar por categoría o mostrar todas.

2.2 Gráfica 2: Popularidad de artículos

Curva mostrando los artículos ordenados de mayor a menor popularidad (número de reviews). Permite filtrar por categoría o todas.

2.3 Gráfica 3: Histograma por nota

Histograma con el número de reviews por cada puntuación (1–5). Permite filtrar por categoría, por todas, o por artículo individual (asin). Si el artículo no existe, se informa al usuario.

2.4 Gráfica 4: Evolución temporal por categorías

Muestra la evolución acumulada de reviews a lo largo del tiempo, con una línea por cada categoría.

2.5 Gráfica 5: Reviews por usuario

Histograma donde el eje x indica el número de reviews y el eje y el número de usuarios que han hecho esa cantidad. Se usa escala logarítmica en el eje y.

2.6 Gráfica 6: Nube de palabras

Nube de palabras generada a partir del campo `summary` de MongoDB. Se pide una categoría obligatoriamente (no se permite "todo"). Solo se usan palabras con más de 3 caracteres.

2.7 Gráfica 7: Rating medio por categoría (libre)

Gráfica de barras horizontales mostrando la puntuación media por categoría. Es distinta a las anteriores ya que muestra promedios, no conteos.

3 Parte 3: Neo4J

El fichero `neo4JProyecto.py` implementa un menú con 4 apartados y una opción de salir.

3.1 Apartado 4.1: Similitudes Pearson

Se obtienen los TOP_N usuarios con más reviews (configurable desde `configuracion.py`, por defecto 30). Se calcula la correlación de Pearson entre cada par de usuarios usando solo los artículos puntuados en común. Se crean nodos de tipo `Usuario` en Neo4J y relaciones `SIMILAR` con la similitud como propiedad. Se muestra por pantalla el usuario con más vecinos.

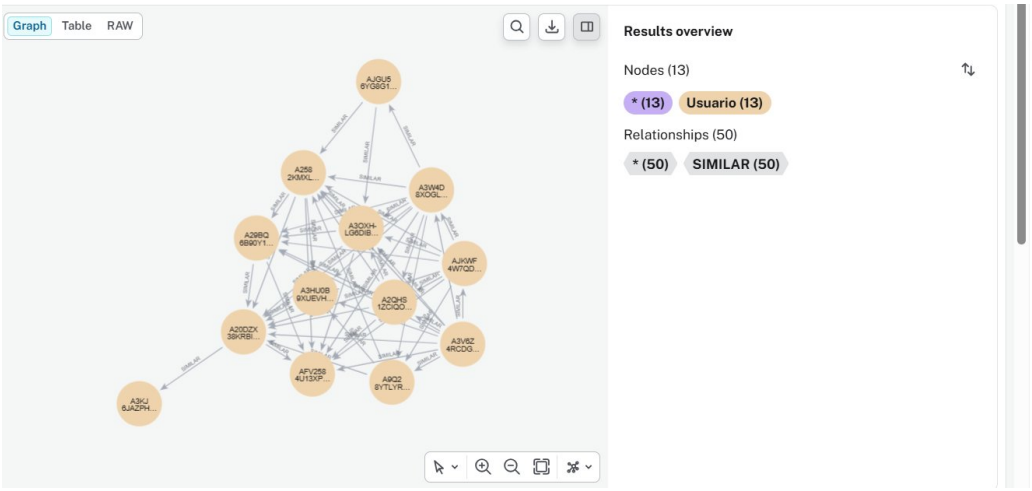


Figure 2: Apartado 4.1: Similitudes Pearson entre usuarios en Neo4J.

3.2 Apartado 4.2: Artículos aleatorios

Se pide al usuario una categoría y un número de artículos. Se seleccionan artículos aleatorios y se cargan en Neo4J junto con todos los usuarios que los han puntuado. Los enlaces incluyen nota y fecha como propiedades.

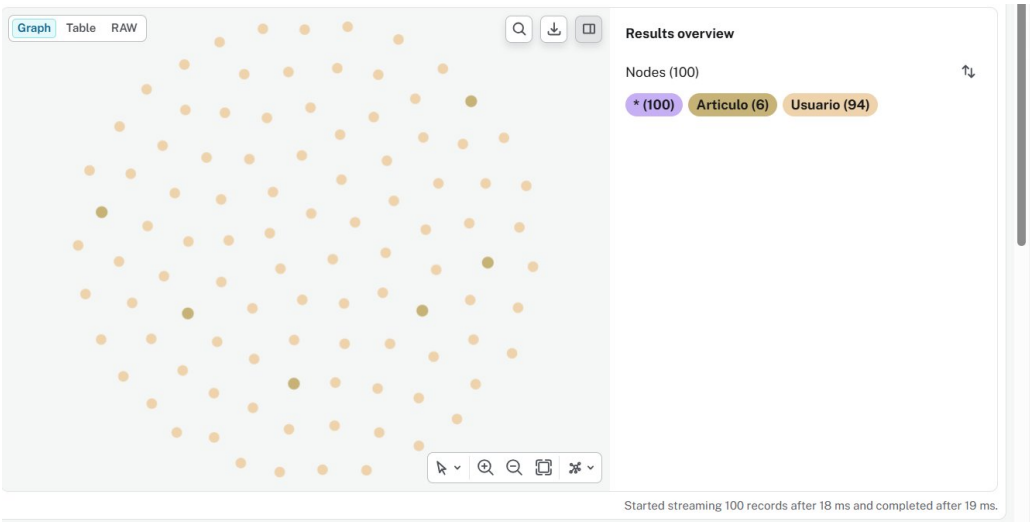


Figure 3: Apartado 4.2: Artículos aleatorios con sus usuarios en Neo4J.

3.3 Apartado 4.3: Usuarios con múltiples categorías

Se seleccionan los primeros 400 usuarios ordenados por nombre. Para los que han puntuado artículos de 2 o más categorías distintas, se crean nodos de `Usuario` y `TipoArticulo` con relaciones que indican la cantidad de artículos consumidos de cada tipo.

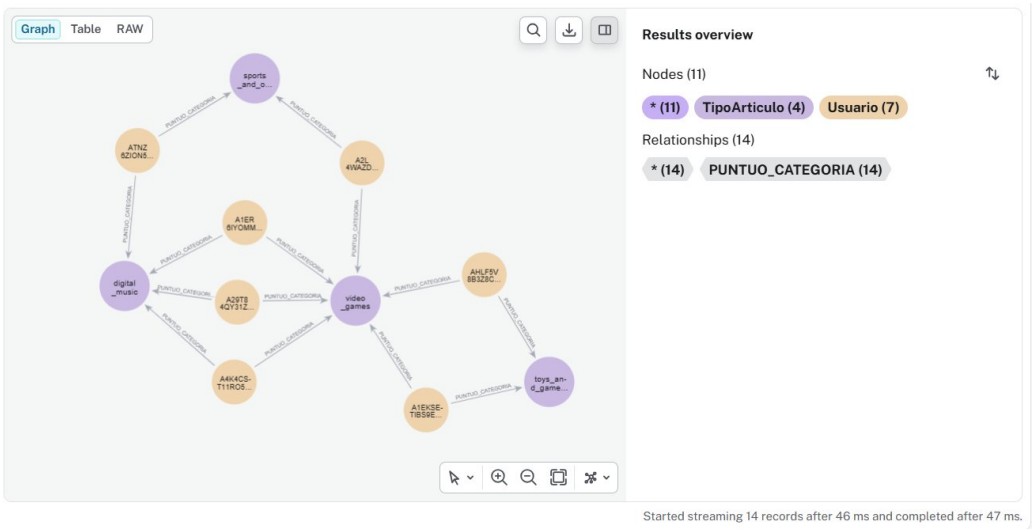


Figure 4: Apartado 4.3: Usuarios con múltiples categorías en Neo4J.

3.4 Apartado 4.4: Artículos populares

Se seleccionan los 5 artículos más populares con menos de 40 reviews. Se cargan en Neo4J con sus usuarios y se crean relaciones `COMPARTEN_GUSTOS` entre usuarios que han puntuado artículos en común.

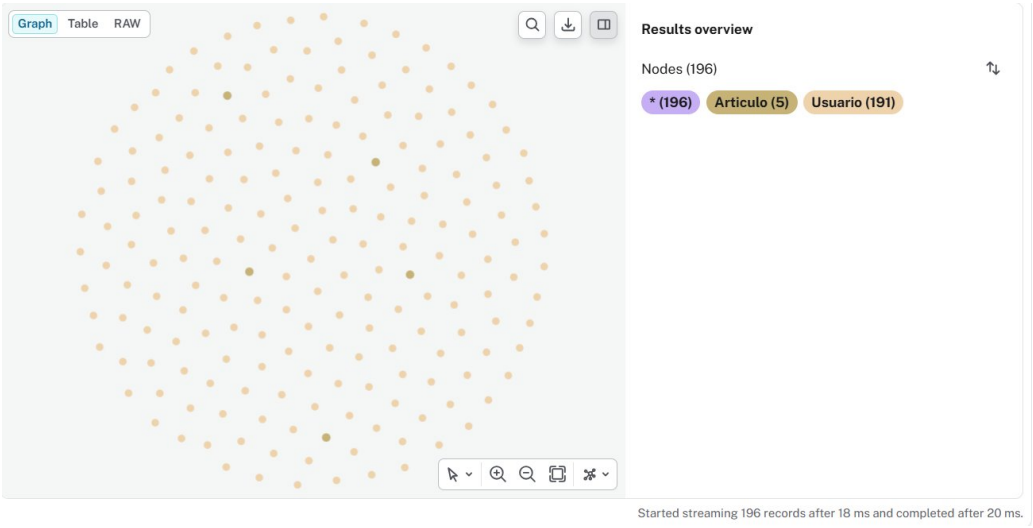


Figure 5: Apartado 4.4: Artículos populares y usuarios en Neo4J.

4 Parte 4: Nuevo dataset

El fichero `inserta_dataset.py` carga un quinto fichero JSON (Sports and Outdoors) en la infraestructura existente, siguiendo el mismo patrón que `load_data.py`:

- Lee la ruta y categoría desde `configuracion.py`.
- Inserta en MySQL y MongoDB con la misma consistencia (DELETE compensatorio si falla MongoDB).
- Comprueba la existencia del usuario y producto antes de insertarlos para evitar duplicados con los 4 datasets originales.

- Lee línea a línea, sin cargar el fichero completo en memoria.
- Convierte `unixReviewTime` a DATE para MySQL y parsea `reviewTime` a fecha antes de insertarlo en MongoDB.

5 Parte 5: Visualización adicional y Machine Learning

5.1 Gráficas en Tableau

Se han reproducido 3 gráficas del menú de Python en Tableau. Los datos se exportaron desde MySQL a CSV y se importaron en Tableau.

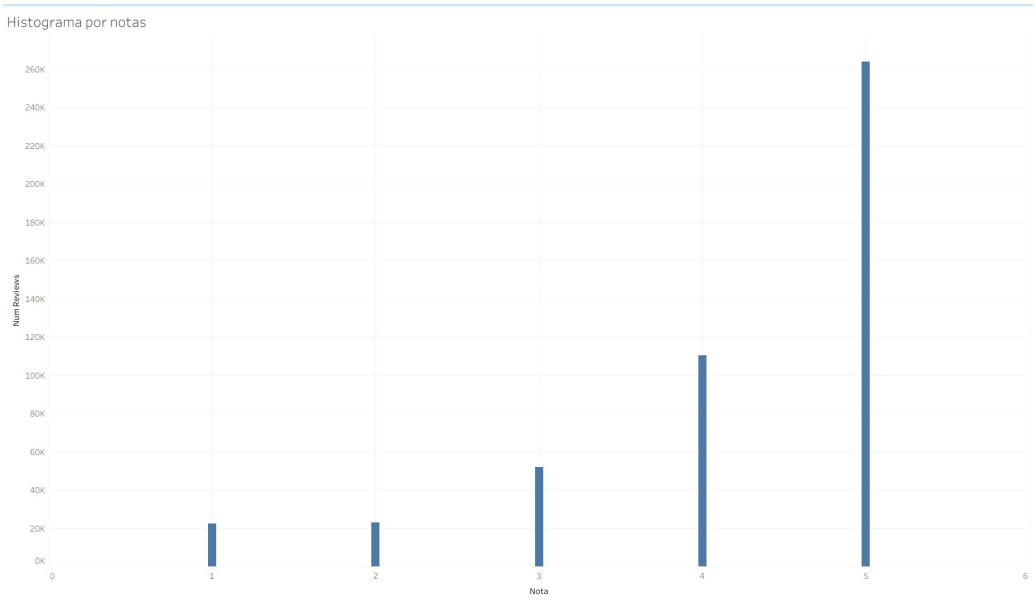


Figure 6: Tableau: Histograma por nota (overall).

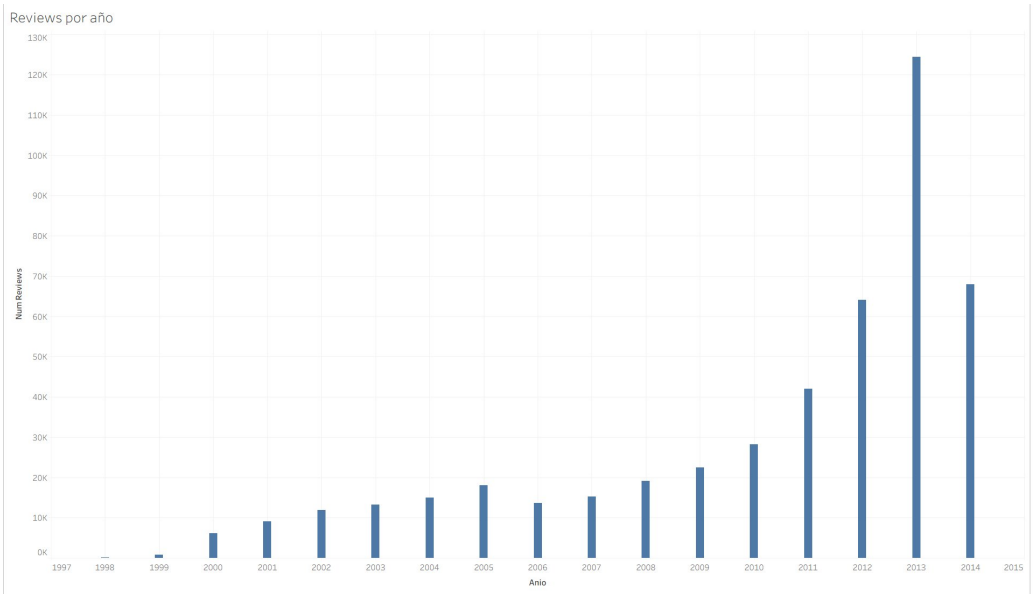


Figure 7: Tableau: Reviews por año.

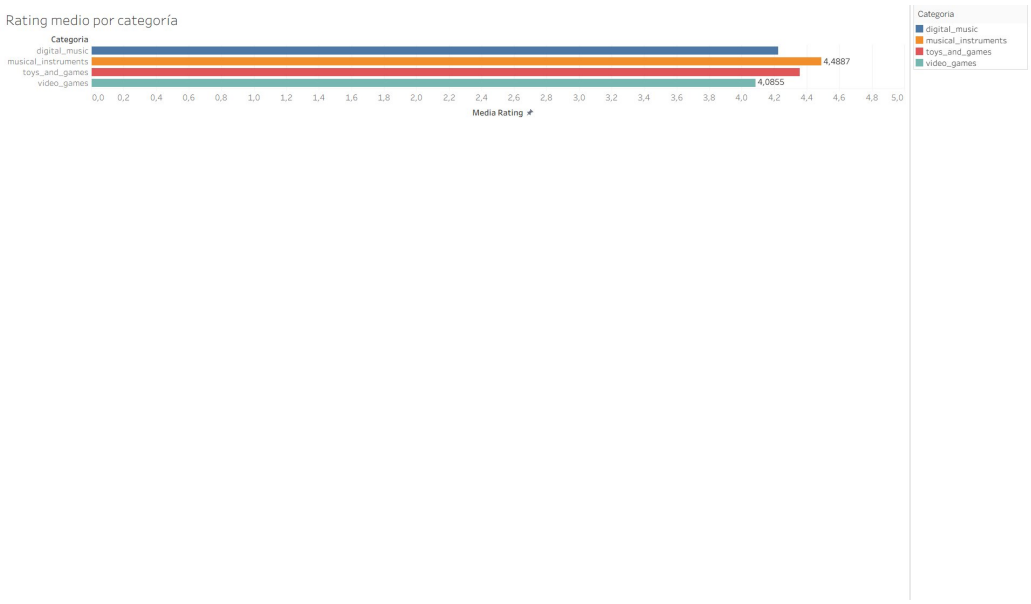


Figure 8: Tableau: Rating medio por categoría.

5.2 Modelo de Machine Learning para recomendación

En este apartado se propone un sistema de recomendación basado en **filtrado colaborativo entre usuarios** (User-Based Collaborative Filtering) como diseño de referencia para la parte de Machine Learning.

Datos de entrada: De MySQL obtenemos la matriz de valoraciones (cada fila es un usuario, cada columna un artículo, cada celda la puntuación). De MongoDB podríamos obtener el texto de las reviews para enriquecer el modelo con análisis de sentimiento.

Fases del modelo:

1. **Construcción de la matriz usuario-artículo:** Cada usuario tiene sus valoraciones. Las celdas vacías representan artículos no consumidos.

2. **Cálculo de similitudes entre usuarios:** Se usa la correlación de Pearson (ya implementada en el apartado 4.1).

3. **Predicción de valoraciones:** Para predecir qué nota daría un usuario U a un artículo A no consumido, se buscan los K usuarios más similares que sí hayan puntuado A. La predicción es una media ponderada por la similitud.

4. **Generación de recomendaciones:** Se predicen valoraciones para todos los artículos no consumidos, se ordenan de mayor a menor y se devuelven los N mejores.

A continuación se muestra el diagrama del modelo propuesto:

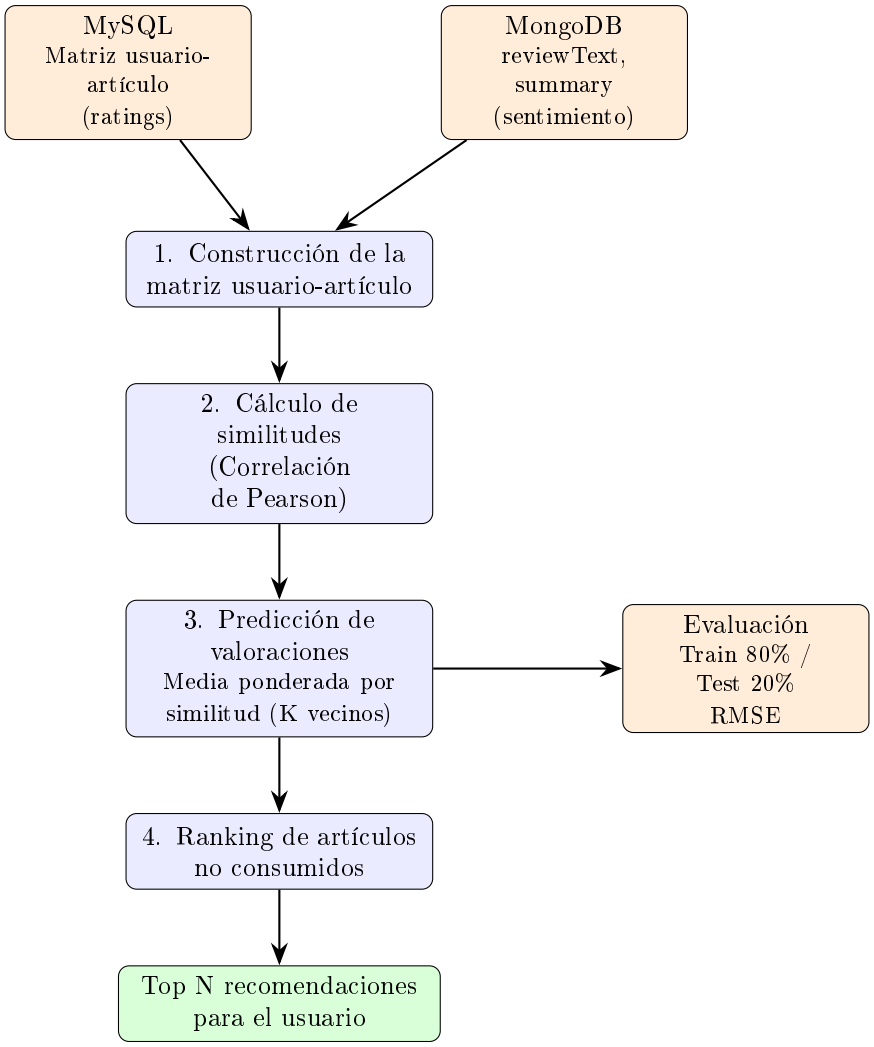


Figure 9: Diagrama del modelo de recomendación basado en filtrado colaborativo.

Evaluación: Se divide el conjunto de reviews en 80% entrenamiento y 20% test. Se comparan las predicciones con las valoraciones reales usando RMSE (error cuadrático medio).

Este diagrama describe el modelo propuesto a nivel conceptual. La implementación actual de `modelo_ml.py` trabaja con los ratings almacenados en MySQL, selecciona a los 200 usuarios más activos, divide los datos en entrenamiento y prueba y evalúa el rendimiento mediante RMSE.

5.3 Mecanismo de recomendación implementado

El fichero `recomendacion.py` implementa un mecanismo que, dado el identificador de un usuario y un tipo de artículo, devuelve los 10 artículos más populares (con más reviews) de esa categoría que el usuario no ha consumido. La consulta SQL usa `NOT IN` para excluir los artículos ya puntuados por el usuario.

6 División del trabajo

Tarea	**Responsable**
Diseño del esquema relacional	Ambos
load_data.py (carga de datos)	Yago Vallarino Fernández-Cavada
configuracion.py	Ambos
menu_visualizacion.py (gráficas)	Yago Vallarino Fernández-Cavada
neo4JProyecto.py (Neo4J)	Yago Vallarino Fernández-Cavada
inserta_dataset.py (5º dataset)	Yago Vallarino Fernández-Cavada
menu_interactivo.py (opcional)	Ignacio Mascarell Agúndez
recomendacion.py (parte 5)	Ignacio Mascarell Agúndez
Gráficas en Tableau	Ignacio Mascarell Agúndez
Modelo de ML (modelo_ml.py)	Ignacio Mascarell Agúndez
Memoria e informe	Ambos

7 Uso de la Inteligencia Artificial

En el siguiente párrafo se explica las ayudas proporcionadas por la Inteligencia Artificial a lo largo de este proyecto. Antes de profundizar, es necesario aclarar que todo el trabajo que se entrega ha sido desarrollado por los integrantes del grupo, Yago Vallarino Fernández-Cavada e Ignacio Mascarell Agúndez, y es únicamente obra suya.

En primer lugar, la IA nos ha ayudado a corregir errores de programación de manera general. Hemos utilizado la inteligencia artificial para que nos resolviese dudas que nos han ido surgiendo debido a fallos de programación. También nos ha guiado en la estructuración de los códigos para lograr unos archivos más claros y eficientes.

Por otro lado, la IA nos ha resuelto dudas generales que nos han surgido en el proyecto. Nos hemos apoyado en la inteligencia artificial en campos como el machine learning, para comprobar que nuestro esquema relacional era coherente o para construir las gráficas con Tableau.

Finalmente, el campo donde más nos hemos servido de la inteligencia artificial es la creación de gráficos y representaciones a través de librerías de Visual Studio Code con matplotlib y streamlit. La inteligencia artificial nos ha guiado para cuadrar las gráficas y nos ha explicado la realización de un menú interactivo para el apartado opcional. Este es sin duda el aspecto donde más ayuda hemos necesitado debido a que es el que menos dominamos.

En conclusión, para la realización de este proyecto nos hemos apoyado en la inteligencia artificial en aspectos como: resolución de dudas generales, ayuda con fallos de programación y representación gráfica. Sin embargo, la IA ha sido una herramienta de ayuda y el trabajo es completamente obra de los integrantes del grupo.